

## 24: Large Language Models and Transformers

*Date: April 21, 2026*

**Acknowledgements.** The notes are built off several resources including Jurafsky SLP3 Ch 10, CS224n Readings, CS324 Winter 2022 notes, and the interactive visualization at Transformer Explainer.

So far we have mostly focused on generative models for vision applications. Today we will look at what generative modelling in the context of language looks like. We will focus mostly on autoregressive language models powered by Transformer architectures. This includes popular GPT models including ChatGPT, and are undoubtedly the most exciting machine learning models these days!

## 1 Language Models

The classic definition of a language model (LM) is a probability distribution over sequences of tokens. For vocabulary  $\mathcal{V}$  of a set of tokens, a LM  $p$  assigns each sequence of tokens  $x_1, \dots, x_T \in \mathcal{V}$  a probability:

$$p(x_1, \dots, x_T).$$

This might look simple, but learning  $p$  requires not only learning grammar of the language, but also semantics (world knowledge). For example, “*The cat ate the fish*” should have much higher probability than “*The fish ate the cat*”.

### 1.1 Autoregressive LMs

Using Bayes Rule, we know that

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t | x_{<t}).$$

Autoregressive LMs are LMs that learn the conditional distribution  $p(x_t | x_{<t})$ .

Now observe that, to sample from this autoregressive models, we can follow the sequential process: first sample  $x_1$ , then sample  $x_2$  conditioned on  $x_1$  and so on. Generally sampling is done with a temperature parameter, that is,

$$x_t \sim p(x_t | x_{<t})^{1/\tau}$$

Here  $\tau = 0$  implies that we choose the most probable token,  $\tau = 1$  implies we choose according to  $p$ , and  $\tau \rightarrow \infty$  implies we choose uniformly from the vocabulary. This allows us to control how much entropy we want from our sampling.

---

<sup>1</sup>Technically we need to renormalize the distribution so it is a probability distribution.

Beyond sampling from the distribution of all possible sequences, we can also perform *conditional sampling* by specifying a prefix  $x_1, \dots, x_k$  (also called prompt) and getting a completion  $x_{k+1}, \dots, x_T$ . This unlocks a lot of capabilities, such as question-answering by choosing the prompt to be a question.

## 1.2 $n$ -grams

One of the earliest LMs were  $n$ -grams dating back to the 1970s. A simplifying assumption we can make is that  $p(x_t|x_{<t}) = p(x_t|x_{t-n-1}, \dots, x_{t-1})$ , that is, the next token depends on only the last  $n - 1$ -tokens. Now, we can use large amounts of text available to compute these probabilities, in particular we need to compute the number of occurrences of all possible  $|\mathcal{V}|^n$  consecutive  $n$  tokens (with appropriate smoothing).

Fitting  $n$ -grams is computationally cheap, but they are pretty limited in their ability to be good LMs. In particular, if we choose  $n$  to be too small then we won't be able to capture long-range dependencies (e.g., predicting the verb 'is' in 'The author, who wrote several complex novels spanning decades, \_\_\_\_\_' based on the singular noun 'author' far back in the context), however, if  $n$  is too large then we will not be able to get good estimates of probabilities, since most long sequences will have nearly 0 probability of showing up in the training set.

## 1.3 Neural LMs

As we have done before many times, instead of computing probabilities based on counts, we can instead let the conditional probability be given by a neural network. Now we can use our data to estimate the parameters  $\theta$  of the neural network by maximizing likelihood of the training data

$$\log p_\theta(x_1, \dots, x_T) = \sum_{t=1}^T \log p_\theta(x_t|x_{<t}).$$

This leads us to the Transformer architecture.

# 2 The Transformer Architecture

In this section we will introduce the architecture of the transformer, which underlies most modern NLP systems. Transformers [VSP<sup>+</sup>17] are sequence-to-sequence models that take in a sequence of  $T$  tokens  $x_1, \dots, x_T \in \mathcal{V}$  and output a sequence of  $T$  contextual embeddings  $z_1^{(L)}, \dots, z_T^{(L)} \in \mathbb{R}^d$  which capture the "contextual" information in the input sequence after  $L$  layers. The key new component of these architectures is the "self-attention" layer.

## 2.1 Input and Position Encodings

The input tokens  $x_1, \dots, x_T \in \mathcal{V}$  are first converted into dense vector representations (embeddings).

1. **Tokenization:** The input text is broken down into tokens (words or subwords) based on a fixed vocabulary  $\mathcal{V}$  (e.g., GPT-2 uses a vocabulary of 50,257 tokens).

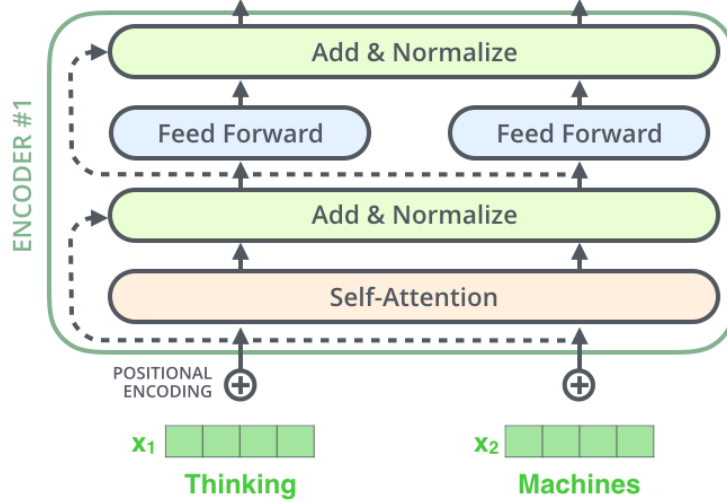


Figure 1: Transformer block overview (Post-Layer Normalization variant). Source: <https://jalammr.github.io/illustrated-transformer/>

2. **Token Embedding:** Each token  $x_i$  is mapped to a vector using an embedding matrix  $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ . Let  $e_i = E(x_i)$  be the embedding for token  $x_i$ .
3. **Positional Encoding:** Since the basic Transformer architecture doesn't inherently process sequences in order (it's permutation-equivariant), information about the position of each token must be added. Without positional information, the model would treat "The cat ate the fish" and "The fish ate the cat" identically after the initial embedding, losing crucial word order meaning. This is typically done by adding a positional encoding vector  $p_i \in \mathbb{R}^d$  to the token embedding  $e_i$ . The initial input representation for token  $i$  is  $x_i^{(0)} = e_i + p_i$ . The positional encodings  $p_i$  can be learned or fixed (e.g., using sine and cosine functions of different frequencies).

Let  $X^{(0)} \in \mathbb{R}^{T \times d}$  be the matrix of these initial representations  $x_1^{(0)}, \dots, x_T^{(0)}$  stacked as row vectors. This matrix  $X^{(0)}$  is the input to the first Transformer block.

## 2.2 Transformer Block

The core of the Transformer is the Transformer block, which is repeated  $L$  times (e.g.,  $L = 12$  times in GPT-2 small). Each block  $l$  (for  $l = 1, \dots, L$ ) refines the sequence representation  $Z^{(l-1)}$  from the previous block (with  $Z^{(0)} = X^{(0)}$  being the initial embeddings) into a new representation  $Z^{(l)}$ . A block consists of two main sub-layers: Multi-Head Self-Attention (MHA) and a Position-wise Feed-Forward Network (FFN), with residual connections and layer normalization (LayerNorm) applied around each sub-layer (see Figure 1).

### 2.2.1 Multi-Head Self-Attention (MHA)

Self-attention allows each token to look at other tokens in the input sequence (including itself) to compute its updated representation. Let's focus on the operations within a single layer  $l$ .

The computation is performed independently by  $N_h$  attention “heads”. We first describe the computation for a single head (suppressing the head index  $h$  and layer index  $l$  on weights temporarily for clarity).

Given the input to the block  $Z^{(l-1)} \in \mathbb{R}^{T \times d}$ , for each token representation  $z_i^{(l-1)}$  (row  $i$ ), we compute three vectors: a Query  $q_i$ , a Key  $k_i$ , and a Value  $v_i$ . These are obtained via linear projections using learned weight matrices  $W_q, W_k, W_v \in \mathbb{R}^{d \times d_k}$  that are specific to this layer  $l$  and this particular head  $h$  (so formally  $W_q^{(l,h)}, W_k^{(l,h)}, W_v^{(l,h)}$ ):

$$q_i = z_i^{(l-1)} W_q, \quad k_i = z_i^{(l-1)} W_k, \quad v_i = z_i^{(l-1)} W_v.$$

(Here  $d_k = d/N_h$  is the dimension per head.) The Transformer Explainer website [CKK<sup>+</sup>22] offers a helpful web search analogy:

- **Query (Q)**: Like the search text you type – the token asking for information.
- **Key (K)**: Like the titles of web pages in results – represents tokens that might be relevant.
- **Value (V)**: Like the content of those web pages – the actual information provided by relevant tokens.

The attention mechanism computes how much token  $i$  (query  $q_i$ ) should attend to token  $j$  (key  $k_j$ ) based on scaled dot-product **attention scores**  $\text{scores}_{ij}$ :

$$\text{scores}_{ij} = \frac{q_i k_j^\top}{\sqrt{d_k}}$$

These attention scores  $\text{scores}_{ij}$  are normalized using softmax across all source tokens  $j$  to obtain attention weights  $\alpha_{ij}$ :

$$\alpha_{ij} = \frac{\exp(\text{scores}_{ij})}{\sum_{j'=1}^T \exp(\text{scores}_{ij'})}.$$

The output vector  $o_i$  for token  $i$  from this head is a weighted sum of the value vectors:

$$o_i = \sum_{j=1}^T \alpha_{ij} v_j.$$

**Matrix Form per Head.** Let  $Z = Z^{(l-1)}$ . The computations for a specific head  $h$  in layer  $l$  can be expressed concisely. Define  $Q^{(l,h)} = ZW_q^{(l,h)}$ ,  $K^{(l,h)} = ZW_k^{(l,h)}$ , and  $V^{(l,h)} = ZW_v^{(l,h)}$ . The output matrix  $O^{(l,h)} \in \mathbb{R}^{T \times d_k}$  for this head is:

$$O^{(l,h)} = \text{Attention}(Q^{(l,h)}, K^{(l,h)}, V^{(l,h)}) = \text{softmax} \left( \frac{Q^{(l,h)} (K^{(l,h)})^\top}{\sqrt{d_k}} \right) V^{(l,h)}.$$

**Multi-Head Combination.** The outputs of all  $N_h$  heads ( $O^{(l,1)}, \dots, O^{(l,N_h)}$ ) are concatenated and then projected back to the model dimension  $d$  using another learned weight matrix  $W_O^{(l)} \in \mathbb{R}^{d \times d}$  specific to layer  $l$ :

$$O_{\text{MHA}}^{(l)} = \text{Concat}(O^{(l,1)}, \dots, O^{(l,N_h)})W_O^{(l)}.$$

**Causal Masking.** In autoregressive models, to prevent attending to future tokens, a mask is applied before the softmax step inside the Attention function (within each head). The mask sets the attention scores  $\text{scores}_{ij}$  corresponding to  $j > i$  to  $-\infty$ . This ensures the corresponding attention weights  $\alpha_{ij}$  become 0.

### 2.2.2 Position-wise Feed-Forward Network (FFN)

The intermediate representation  $H^{(l)}$  is then passed through a position-wise FFN. This network is applied independently to each token representation (each row of  $H^{(l)}$ ). It usually consists of two linear transformations with layer-specific weights  $W_1^{(l)}, W_2^{(l)}$  and biases  $b_1^{(l)}, b_2^{(l)}$ , with a non-linear activation function applied after the first transformation. Common choices for the activation include ReLU (Rectified Linear Unit) or GeLU (Gaussian Error Linear Unit):<sup>2</sup>

$$\text{FFN}(h) = \text{Activation}(hW_1^{(l)} + b_1^{(l)})W_2^{(l)} + b_2^{(l)} \quad (\text{e.g., Activation} = \text{ReLU or GeLU})$$

where  $h$  represents a single row (token representation) from  $H^{(l)}$ ,  $W_1^{(l)} \in \mathbb{R}^{d \times d_{ff}}$ ,  $b_1^{(l)} \in \mathbb{R}^{d_{ff}}$ ,  $W_2^{(l)} \in \mathbb{R}^{d_{ff} \times d}$ ,  $b_2^{(l)} \in \mathbb{R}^d$ , and  $d_{ff}$  is the inner dimension (often  $d_{ff} = 4d$ ). Let  $O_{\text{FFN}}^{(l)} = \text{FFN}(H^{(l)})$  denote the result of applying the FFN row-wise to the entire matrix  $H^{(l)}$ . This expansion to a larger intermediate dimension  $d_{ff}$  allows the model to potentially learn richer and more complex combinations of features for each token individually after information has been mixed across tokens by the self-attention layer.

### 2.2.3 Residual Connections and Layer Normalization

To facilitate the training of deep networks and stabilize activations, each sub-layer (MHA and FFN) is wrapped by a residual connection followed by layer normalization (Post-LN).

**Layer Normalization** [BKH16] normalizes the inputs across the features independently for each token. For an input vector  $x \in \mathbb{R}^d$  at a specific token position, the mean  $\mu$  and variance  $\sigma^2$  are calculated across the feature dimension  $d$ . The normalization is then:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad \text{where } \mu = \frac{1}{d} \sum_{i=1}^d x_i, \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

Here  $\gamma, \beta \in \mathbb{R}^d$  are learnable scale and shift parameters, and  $\epsilon$  is for numerical stability.

---

<sup>2</sup>ReLU is defined as  $\text{ReLU}(x) = \max(0, x)$ . GeLU is defined as  $\text{GeLU}(x) = x\Phi(x)$ , where  $\Phi(x)$  is the standard Gaussian cumulative distribution function (CDF). Fast approximations are often used in practice, e.g.,  $\text{GeLU}(x) \approx x\sigma(1.702x)$  where  $\sigma$  is the sigmoid function.

### 2.2.4 Putting it all together

The sequence of operations for a full block  $l$  is:

1. MHA Output:  $O_{\text{MHA}}^{(l)} = \text{MHA}(Z^{(l-1)})$
2. First Add and Norm:  $H^{(l)} = \text{LayerNorm}(Z^{(l-1)} + O_{\text{MHA}}^{(l)})$
3. FFN Output:  $O_{\text{FFN}}^{(l)} = \text{FFN}(H^{(l)})$
4. Second Add and Norm (Block Output):  $Z^{(l)} = \text{LayerNorm}(H^{(l)} + O_{\text{FFN}}^{(l)})$

This final  $Z^{(l)}$  becomes the input for the next block  $(l + 1)$ .

## 2.3 Output Layer and Next-token Prediction

After the input has passed through all  $L$  Transformer blocks, we obtain the final sequence of contextual embeddings  $Z^{(L)} = (z_1^{(L)}, \dots, z_T^{(L)})$ . For autoregressive language modeling (predicting the next token), we use the output embedding  $z_t^{(L)}$  corresponding to the  $t$ -th token to predict the  $(t + 1)$ -th token  $x_{t+1}$ .

This is done by projecting  $z_t^{(L)}$  back into the vocabulary space using a final linear layer (often implemented by multiplying with the transpose of the input embedding matrix  $E^\top$ , i.e.,  $W_{\text{out}} = E^\top$ ) to get scores (logits) for each word in the vocabulary:

$$\text{logits}_t = z_t^{(L)} W_{\text{out}} \quad (\text{where } W_{\text{out}} \in \mathbb{R}^{d \times |\mathcal{V}|})$$

These logits are then converted into probabilities using the softmax function:

$$p(x_{t+1}|x_{1:t}) = \text{softmax}(\text{logits}_t)$$

This gives a probability distribution over the entire vocabulary  $\mathcal{V}$  for the next token.

## 2.4 Sampling Strategies

To generate text, we start with a prompt (or a start-of-sequence token). At each step  $t$ :

1. Feed the current sequence  $x_{1:t}$  through the Transformer to get the final output embeddings  $Z^{(L)}$ .
2. Compute the probability distribution for the next token  $p(x_{t+1}|x_{1:t})$  using  $z_t^{(L)}$  and the output layer.
3. Sample the next token  $\hat{x}_{t+1}$  from the computed distribution  $p_t = p(x_{t+1}|x_{1:t})$ . Several techniques can be used to control the sampling process:

- **Temperature Scaling:** As introduced in Section 1.1, the temperature  $\tau$  controls the randomness of the sampling. Specifically, the logits  $l$  (vector of scores for each word in  $\mathcal{V}$ ) are divided by  $\tau$  before the softmax calculation:

$$p'(x_{t+1} = w \mid x_{1:t}) = \frac{\exp(l(w)/\tau)}{\sum_{w' \in \mathcal{V}} \exp(l(w')/\tau)}$$

Lower values of  $\tau$  (e.g.,  $< 1$ ) make the output more deterministic (closer to greedy), while higher values (e.g.,  $> 1$ ) increase diversity.

- **Top-k Sampling:** Instead of considering the full vocabulary, restrict the sampling pool to only the  $k$  tokens with the highest probabilities according to  $p_t$ . Then, renormalize the probabilities among these  $k$  tokens and sample from this smaller distribution. This avoids sampling very unlikely tokens while still allowing some diversity among the top choices.
- **Top-p (Nucleus) Sampling:** Select the smallest set of tokens whose cumulative probability exceeds a threshold  $p$  (e.g.,  $p = 0.9$ ). Sort tokens by probability, add them to the set one by one until the cumulative probability reaches  $p$ , then renormalize and sample from this set (the “nucleus”). This method adaptively chooses the number of tokens to consider based on the shape of the distribution – if the model is very confident (sharp distribution), the nucleus is small; if uncertain (flat distribution), the nucleus is larger.

These techniques (often Temperature + Top-k or Temperature + Top-p) help balance coherence and diversity in the generated text.

4. Append  $\hat{x}_{t+1}$  to the sequence and repeat until an end-of-sequence token is generated or a maximum length is reached.

## 2.5 Training Objective

Transformers are typically trained using Maximum Likelihood Estimation (MLE) on large text corpora. The objective is to maximize the log-probability of the true next token  $x_t$  given the preceding context  $x_{<t}$ , summed over all positions in all training sequences:

$$\mathcal{L}(\theta) = \sum_{\text{sequences } x} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})$$

This loss is minimized using variants of stochastic gradient descent, like Adam [KB14]. This training process is a form of self-supervised learning, as the labels (the next tokens) are derived directly from the input sequence itself, requiring no external annotation. A key advantage of the Transformer architecture is its high parallelizability. Within each layer, computations across the sequence length  $T$  (especially in the FFN) and across different attention heads can be performed in parallel. This structure maps extremely well to modern hardware accelerators like GPUs and TPUs, which excel at the large matrix multiplications involved, enabling the training of very large models. However, the quadratic complexity of the self-attention mechanism with respect to sequence length ( $O(T^2d)$ ) can become a significant bottleneck for very long sequences ( $T$ ). This arises from the need to explicitly compute, store, and process the full  $T \times T$  attention matrix (involving score calculation, softmax, and value aggregation), which inherently requires considering all pairs of token interactions.

## 2.6 Parameter Distribution and Scaling

Understanding where the parameters reside gives insight into the model’s structure and scaling properties. In a typical Transformer, the largest portion of parameters (often 45-50%) is found in the Feed-Forward Network (FFN) layers across all blocks, driven largely by the intermediate dimension  $d_{ff}$  (typically  $4d$ ). The token embedding matrix (including the tied output projection) constitutes the next major share (around 30-35%) of the parameters, though this is heavily dependent on the vocabulary size  $|\mathcal{V}|$ . The weight matrices associated with the multi-head attention mechanism itself account for the remaining significant portion (around 20-25%) of the parameters. Parameters for positional embeddings (if learned) and layer normalization are usually negligible in comparison.

Therefore, while attention is the core mechanism, the bulk of the parameters in standard Transformers reside in the FFN layers and the token embedding matrices. Scaling up these models involves increasing the number of layers  $L$ , the model dimension  $d$ , the vocabulary size, and/or the FFN intermediate dimension  $d_{ff}$ .

## 3 Large Language Models

We have so far discussed language models and the Transformer architecture, which forms the basis for modern Large Language Models (LLMs). For example, GPT-3 [BMR<sup>+</sup>20] has 175 billion parameters and was trained on hundreds of gigabytes of text data. GPT-4 is estimated to be even larger (over a trillion parameters).

A lot of compute resources across companies have gone towards scaling these models to make them bigger. The interesting observation is that new emergent behavior appears with scale [WTB<sup>+</sup>22], leading to qualitatively different capabilities and qualitatively different societal impact. These include the ability to be a standalone system to solve all sorts of tasks like question-answering, summarization, story generations, etc. One surprising ability that GPT-3 has is in-context learning. This is the ability to give a few examples of solving a problem in context, and then the model solving them for subsequent examples.

However, with increasing capabilities, the risks posed by these models also increase. These models often hallucinate knowledge (produce false facts), propagate social bias such as gender stereotypes, produce toxic content, can be attacked to reproduce private information from the training data, and more. Furthermore, the environmental and computational costs of these models are very high, and most LLMs are closed and not accessible. These are hard problems without many answers that are a focus of many research problems today.

## References

- [BKH16] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [BMR<sup>+</sup>20] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.



- [CKK<sup>+</sup>22] Aeree Cho, Grace C. Kim, Alexander Karpekov, Alec Helbling, Jay Wang, Seongmin Lee, Benjamin Hoover, and Polo Chau. Transformer explainer. url<https://poloclub.github.io/transformer-explainer/>, 2022. Accessed: April 2025.
- [KB14] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [VSP<sup>+</sup>17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [WTB<sup>+</sup>22] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.